

TUGAS PEMROGRAMAN DASAR

Kelompok : Obsidian

Nama : Rinoa Annisa Permata

Problem 1

1. Pseudocode :

PROGRAM_The_Last_Astronaut

DECLARE

N, K : integer
astronaut : list of integer
index : integer
posisi : integer
tereliminasi : integer

BEGIN

Input N

Input K

astronaut $\leftarrow$ [1, 2, 3, ..., N]

index $\leftarrow$ 0

WHILE jumlah astronaut > 1 DO

// Menentukan posisi astronaut yang dieliminasi

posisi $\leftarrow$ (index + K - 1) MOD jumlah astronaut

tereliminasi $\leftarrow$ astronaut[posisi]

// Tampilkan/simpan urutan eliminasi Output tereliminasi

// Hapus astronaut dari lingkaran

Hapus astronaut[posisi]

// Mengubah nilai K

IF tereliminasi MOD 2 = 0 THEN

K $\leftarrow$ K + 2

ELSE

$K \leftarrow K - 1$

END IF

// K tidak boleh kurang dari 2

IF $K < 2$ THEN

$K \leftarrow 2$

END IF

// Astronaut berikutnya berada pada posisi yang sama

// karena elemen setelah yang dihapus bergeser ke posisi tersebut

IF jumlah astronaut > 0 THEN

Index $\leftarrow$ posisi MOD jumlah astronaut

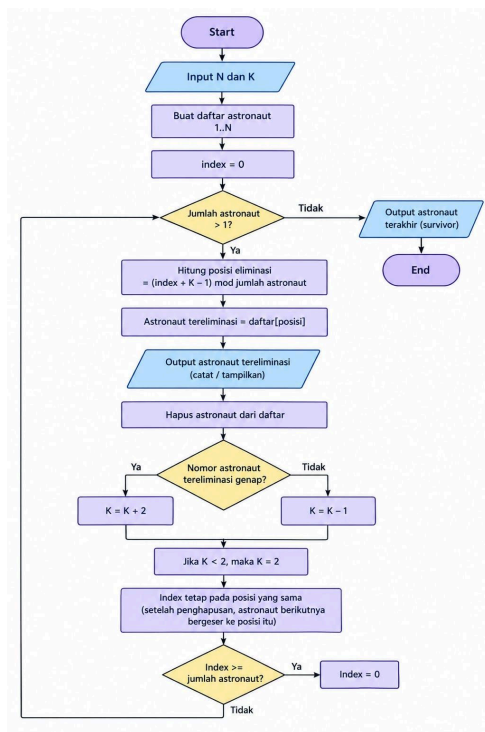
END IF

END WHILE

Output "Astronaut terakhir: ", astronaut[0]

END

2. Flowchart :



Penjelasan logika :

Logika pada kasus The Last Astronaut menggunakan konsep eliminasi berulang pada sekumpulan astronaut yang disusun membentuk lingkaran. Pada awal proses, astronaut diberi nomor dari 1 hingga N dan perhitungan dimulai dari astronaut nomor 1. Dengan nilai K yang telah ditentukan, setiap astronaut yang mendapatkan hitungan ke-K akan dieliminasi dari lingkaran. Setelah seorang astronaut dieliminasi, proses perhitungan selanjutnya dimulai dari astronaut yang berada tepat setelah astronaut yang telah dieliminasi. Proses ini dilakukan secara terus-menerus sampai hanya tersisa satu astronaut sebagai astronaut terakhir yang bertahan.

Hal yang membedakan kasus ini adalah nilai K tidak selalu tetap. Setelah setiap eliminasi, nomor astronaut yang tereliminasi diperiksa apakah termasuk bilangan genap atau ganjil. Jika nomor tersebut genap, nilai K bertambah 2, sedangkan jika nomor tersebut ganjil, nilai K berkurang 1. Namun, nilai K tidak boleh kurang dari 2, sehingga apabila hasil perubahan menyebabkan K kurang dari 2, maka K akan diatur kembali menjadi 2. Untuk menentukan posisi astronaut yang akan dieliminasi dalam lingkaran, digunakan operasi modulo sehingga ketika perhitungan mencapai astronaut terakhir, proses dapat kembali ke astronaut pertama. Algoritma terus melakukan proses pencarian posisi, eliminasi, perubahan K, dan pergeseran titik awal sampai tersisa satu astronaut, kemudian nomor astronaut tersebut menjadi hasil akhir.

Problem 2

1. Pseudocode

PROGRAM_Sandi_Alien

DECLARE

 pesan : String

 hasil : string

 nilaiSekarang : integer

 nilaiSebelumnya : integer

 nilaiBaru : integer

 i : integer

BEGIN

input pesan

hasil ← ""

// Huruf pertama tidak berubah

Hasil ← hasil + pesan[0]

For i ← 1 TO panjang(pesan) - 1 DO

 nilaiSekarang ← nilai huruf pesan[i]

 nilaiSebelumnya ← nilai huruf pesan[i-1]

 nilaiBaru ← nilaiSekarang + nilaiSebelumnya

 IF nilaiBaru > 26 THEN

 nilaiBaru ← ((nilaiBaru - 1) MOD 26) + 1

 END IF

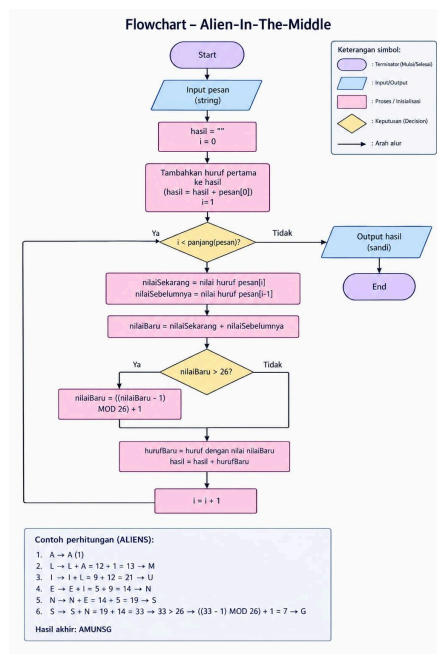
 hasil ← hasil + huruf dengan nilai nilaiBaru

END FOR

Output hasil

SELESAI

2. Flowchart



Penjelasan logika :

Logika pada kasus Alien-In-The-Middle adalah membuat sistem sandi dengan mengubah setiap huruf pada sebuah pesan berdasarkan nilai huruf tersebut dan nilai huruf sebelumnya. Setiap huruf dalam alfabet memiliki nilai tertentu, yaitu $A = 1$, $B = 2$, $C = 3$ hingga $Z = 26$. Huruf pertama pada pesan tidak mengalami perubahan dan langsung menjadi bagian dari hasil sandi. Setelah itu, setiap huruf berikutnya diproses dengan cara menjumlahkan nilai huruf tersebut dengan nilai huruf yang berada tepat sebelumnya pada pesan asli. Hasil penjumlahan kemudian dikonversikan kembali menjadi huruf alfabet.

Apabila hasil penjumlahan lebih dari 26, maka perhitungan harus kembali ke awal alfabet. Hal ini dilakukan menggunakan operasi modulo 26 sehingga nilai setelah Z akan kembali dimulai dari A. Sebagai contoh, pada pesan ALIENS, huruf L memiliki nilai 12 dan huruf sebelumnya A memiliki nilai 1, sehingga menghasilkan 13 atau M. Selanjutnya I bernilai 9 ditambah L yang bernilai 12 menghasilkan 21 atau U. Proses yang sama dilakukan sampai huruf terakhir sehingga diperoleh hasil sandi AMUNSG. Dengan demikian, program harus membaca pesan, memproses setiap huruf sesuai aturan, menangani hasil yang melewati nilai 26, lalu menggabungkan seluruh huruf baru menjadi pesan yang telah disandikan.